

ACLens: Graph-Guided Multi-Agent Framework for Access Control Vulnerability Detection in Smart Contracts

Anonymous Author(s)

Abstract

Smart contracts are immutable, self-executing programs deployed on blockchains that typically manage substantial financial assets, making them a prime target for attackers. While Access Control (AC) vulnerabilities constitute the leading cause of financial loss, identifying them is increasingly challenging as authorization logic spans multiple functions with shared state variables and diverse business rules. Existing detectors heavily dependent on guard pattern matching or sender taint analysis, but these methods frequently prove ineffective in the context of DeFi protocols. The root cause of such vulnerabilities is rarely the absence of guard patterns, but rather the exploitable reachability of privileged operations under unintended policies.

Tackling this fundamental issue requires evaluating two distinct criteria, namely identifying which operations carry privileged effects and determining whether any feasible path can reach them in violation of the authorization policy implied by the contract's own protection patterns. In this work, we propose ACLens, a multi-agent framework guided by program graphs to conduct the analysis in two dedicated phases. The first phase performs deterministic graph analysis to pinpoint privileged sinks and delineate a compact evidence boundary around them. The second phase deploys three agents to reconstruct an execution trace reaching each sink, infer the intended authorization policy from intra-contract evidence, and determine whether the reconstructed trace violates the inferred policy. On a DeFi exploit benchmark of 46 validated projects and 98 vulnerable functions, ACLens reaches 83.5% recall and 55.1% F1, outperforming the strongest baseline (i.e., ACTaint) by 46.4 points in recall and 20.0 points in F1.

Keywords

Access Control Vulnerability, Smart Contracts, Code Property Graph, Large Language Models, Multi-Agent Analysis

1 Introduction

Decentralized finance (DeFi) protocols built on smart contracts now manage billions of dollars in total value locked [? ?]. Because these contracts are immutable once deployed, a vulnerability cannot be patched and exploitation leads to irreversible financial losses [? ?]. Among all vulnerability classes, *access control (AC) defects* are the most costly, arising when critical operations are executed without proper authorization. In 2024, access-control exploits accounted for 75% of all crypto hacks, with roughly \$1.7 billion stolen [?]. The 2025 OWASP Smart Contract Top 10 ranks AC vulnerabilities as the number-one threat [?].

A public function that modifies the owner variable is only vulnerable if the developer intended to limit access to the variable. This

dependence on implicit intent, rather than on language semantics, distinguishes AC from vulnerability classes such as reentrancy and integer overflow. On CVE-style benchmarks where each flaw is confined to a single function, specialized tools perform reasonably well. ACTaint [?] reports 93.3% recall on the CVE dataset, and AChecker [?] reaches 80%. These benchmarks rarely reflect production DeFi contracts where authorization logic spans multiple functions and AC exploits cause the largest financial losses [?]. Once authorization logic spans multiple functions with shared state variables and contract-specific business rules, the assumptions behind single-function analysis no longer hold. In our evaluation on a new DeFi benchmark under a strict function-level protocol (Section 4), the best-performing specialized baseline reaches only 37.1% recall on confirmed exploits.

AC vulnerability detection begins by locating critical operations (e.g., external transfers, privileged state writes, and self-destruction), relying mostly on structural patterns. The primary challenge lies in deciding whether a located sink is actually vulnerable. Traditional tools address this challenge by verifying the presence of known guard patterns. Although some advanced systems [? ?] incorporate dataflow or taint analysis, they mainly rely on a closed-world assumption, validating only against predefined protection mechanisms. In production contracts, however, authorization logic is rarely confined to standard modifiers or guard patterns. It emerges from intra-contract state relationships that are distributed across multiple functions and shared state variables, making it only partially visible to pattern-based analysis.

LLM-based approaches have also been explored for AC detection [?]. Applying them effectively, however, requires overcoming two obstacles inherent to AC vulnerabilities. **C1: Authorization logic is structurally indistinguishable in standard program representations.** In conventional program graphs, guards that depend on the caller are encoded identically to ordinary business predicates, and all accesses to the same indexed state variable collapse into a single abstract location regardless of their index keys. As a result, no structural signal separates an authorization check from an arithmetic bound or a balance lookup. This absence undermines the validity of any analysis relying on such representations. Providing the entire contract to an LLM obscures authorization signals amidst irrelevant code [?], whereas slicing along standard dependence edges fails to preserve cross-function state relationships. In both cases, the pivotal evidence necessary for determining AC vulnerabilities is lost. **C2: Vulnerability is defined against an implicit policy specific to the contract.** Certain common vulnerabilities, such as reentrancy and integer overflow, stem from violations of language-level semantics that can be defined independently of any specific contract. AC violations, by contrast, are meaningful only relative to an authorization policy that the developer rarely specifies explicitly. Traditional detectors reduce this ambiguity to closed-world pattern matching. LLMs can

ASE '26,
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

reason about open-ended policies, but without structured comparative evidence they cannot distinguish intentionally permissionless interfaces from accidentally unprotected ones.

Despite the semantic complexity of access control (AC) rules, their structural evidence can be identified through dependency relationships before any semantic interpretation is required. Building on this insight, we propose ACLens, a multi-agent framework for detecting AC vulnerabilities in smart contracts, guided by program graphs. Concretely, ACLens formulates AC detection as a two-phase process. In the first phase, Graph-Guided Structural Analysis leverages an AC-aware property graph to identify security-critical sinks and extract a compact evidence boundary for each target function. In the second phase, Multi-Agent Security Reasoning decomposes the analysis across specialized agents: one reconstructs the execution trace leading to the sink, another infers the intended authorization policy from existing protection patterns in the contract, and a verifier determines whether the trace violates that policy. Rather than exposing each reasoning agent to the entire contract, ACLens uses dependency graphs to define a per-function evidence boundary prior to any semantic analysis. Our evaluation on a DeFi benchmark comprising 46 validated projects (Section 4) shows that this decomposition achieves 83.5% recall and 55.1% F1, surpassing the strongest specialized baseline by 46.4 in recall and 20.0 in F1 score. In summary, this work makes the following contributions.

- We propose ACLens, a multi-agent framework for AC vulnerability detection in smart contracts that combines graph-guided structural localization with multi-agent policy reasoning. To the best of our knowledge, this is the first work to leverage program graph analysis to delimit the evidence scope for each critical sink, thereby enabling more effective multi-agent reasoning for AC vulnerability detection.
- In ACLens, we design an Access Control Property Graph (ACPG) that extends standard code property graphs with caller-dependent guard semantics and cross-function state links, enabling precise sink identification and compact evidence extraction via backward slicing.
- We evaluate ACLens on a DeFi benchmark of 46 projects spanning six chains. Under a strict function-level protocol over confirmed exploit cases, ACLens achieves 83.5% recall and 55.1% F1, more than doubling the recall of the best-performing specialized baseline (37.1%) while maintaining competitive precision.

2 Motivating Example

We illustrate the limitations of existing AC detection approaches using a real-world exploit from April 2024. The target is XBridge [?], a cross-chain bridge contract deployed on Ethereum. An attacker exploited a scoping inconsistency between token registration and ownership tracking to hijack deposited liquidity.

The vulnerability. The bridge holds tokens deposited by liquidity providers. `listToken` registers a token for cross-chain bridging and records the caller as the token’s owner in `_tokenOwner` (line 17). `withdrawTokens` checks this mapping and transfers funds only to the recorded owner (line 23).

The bug lies in a dimensionality mismatch between two mappings. The uniqueness check on line 11 queries the three-dimensional `pairMap`, indexed by source chain, destination chain,

```
// Chain-pair-specific token mapping
mapping(uint => mapping(uint =>
mapping(address => address))) public pairMap;
// Global token ownership (NOT chain-specific)
mapping(address => address) public _tokenOwner;

// Function A: register a cross-chain token pair
function listToken(TokenInfo memory base,
TokenInfo memory corr, bool mintable)
external payable {
require(pairMap[base.chain][corr.chain]
[base.token] == address(0), "LISTED");
pairMap[base.chain][corr.chain]
[base.token] = corr.token;
isMintable[base.chain][corr.chain]
[base.token][corr.token] = mintable;
_tokenOwner[base.token] = msg.sender;
}

// Function B: withdraw deposited liquidity
function withdrawTokens(address token,
address receiver, uint256 amount) external {
require(msg.sender == _tokenOwner[token],
"ONLY_LISTER");
IERC20(token).transfer(receiver, amount);
}
```

Figure 1: Simplified XBridge cross-chain bridge.

and token address. It therefore prevents the same token from being listed twice on the *same* chain pair. The ownership assignment on line 17, however, writes to the one-dimensional `_tokenOwner`, keyed *solely* by token address with no chain-pair scoping. A token can therefore be listed on multiple chain pairs, and each listing silently overwrites its global owner. An attacker exploits this by re-listing an already-registered token on a different chain pair, hijacking its ownership entry, and draining the deposited liquidity via `withdrawTokens`. Any analysis that examines each function in isolation, or treats all accesses to `_tokenOwner` as equivalent regardless of their key structure, will miss this vulnerability. Existing tools fail for different but complementary reasons.

- **Bytecode-level taint (AChecker [?]).** AChecker performs intraprocedural `msg.sender` dependency analysis on EVM bytecode. It detects the `require(msg.sender == _tokenOwner[token])` guard in `withdrawTokens` and classifies the function as protected. Its single-function analysis scope cannot observe the cross-function overwrite of `_tokenOwner` via `listToken`.
- **LLM-guided taint (ACTaint [?]).** ACTaint’s Contract Variable Dependency Graph (CVDG) tracks dependencies at the variable level and records that both functions access `_tokenOwner`. The CVDG does not resolve mapping keys. It therefore cannot distinguish `_tokenOwner[base.token]` in `listToken` from `_tokenOwner[token]` in `withdrawTokens`. Its taint analysis then classifies the `require` guard as valid protection and misses the ownership-hijacking vector.
- **Standalone LLM (GPT-4o).** Given all 1,600+ lines, GPT-4o’s attention dilutes across dozens of irrelevant functions. In our runs it either hallucinated a non-existent guard on `listToken` or deemed public registration “intentional.”

Detecting this vulnerability demands cross-function state tracking at the mapping-key level, since variable-level dependency alone cannot distinguish the two access patterns. The 1,600 lines of surrounding code must then be distilled to the small subset of

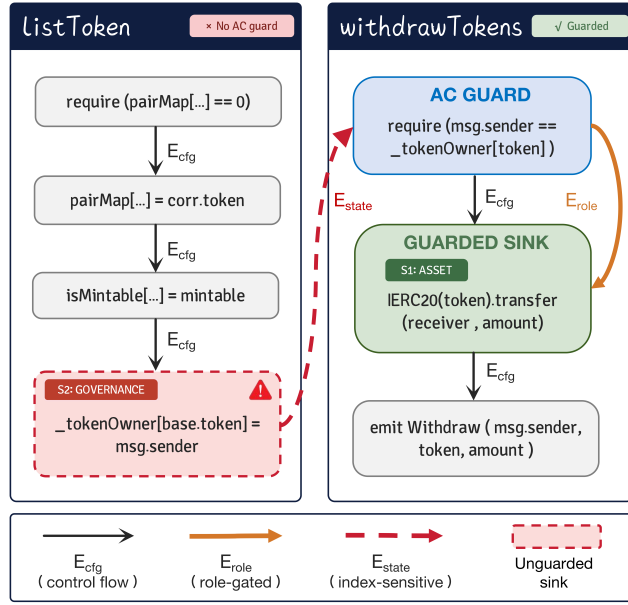


Figure 2: ACPG fragment for the XBridge vulnerability.

statements that carry authorization semantics. The resulting policy judgment has no predefined template to match against. Section 3 describes how ACLens addresses these requirements.

3 ACLens

3.1 Overview

Detecting AC vulnerabilities requires first locating the operations that carry privileged effects and delimiting the security-relevant context around them. It further requires determining whether any feasible execution reaches those operations in violation of the contract’s implicit authorization policy. ACLens delegates these two tasks to structural analysis guided by the program graph and multi-agent security reasoning, respectively. By design, the first phase determines what each agent in the second phase is permitted to see, ensuring that semantic judgment is always grounded in structurally scoped evidence. Figure 3 presents the overall workflow.

Graph-Guided Structural Analysis (§3.2) builds an Access Control Property Graph (ACPG) that captures caller-dependent guard semantics and cross-function state dependencies, and uses them to identify authorization-relevant sinks. Around each sink, relevance-guided slicing then extracts a Sink-Conditioned Execution Environment (SCEE) based on the sink set and graph topology.

Multi-Agent Security Reasoning (§3.3) consists of trace reconstruction, policy inference, and compliance verification. The Trace agent reconstructs a candidate sink-reaching execution trace for each sink. The Auditor infers the intended policy from code structure and graph annotations. The Verifier then aligns these two outputs to judge whether the reconstructed trace violates the inferred policy. The evaluation in §4 quantifies the benefit of this separation.

3.2 Graph-Guided Structural Analysis

Passing the full contract to phase two would obscure authorization logic among irrelevant code, while naive slicing on a standard dependency graph would discard the cross-function state flows that AC judgment depends on. Graph-guided structural analysis addresses both requirements through three steps: ACPG construction, AC-critical sink identification, and relevance-guided context slicing.

3.2.1 ACPG Construction. We first construct a Code Property Graph (CPG) [?] from Slither’s [?] parsing results. A CPG is a directed multigraph whose nodes represent program statements and whose edges encode three relationships. Control-flow edges capture execution order between statements. Data-dependence edges link each variable definition to the statements that read it. Control-dependence edges connect conditional branches to the statements they govern. This unified representation allows downstream analysis to simultaneously trace execution paths, data flows, and branching conditions around a critical operation within one structure.

For AC analysis, two aspects of a standard CPG remain implicit. First, authorization semantics is not distinguished from application logic. For example, predicates that enforce authorization, such as the one in `require(msg.sender == owner)`, are represented in the same way as general-purpose predicates like `amount > 0`. Second, a standard CPG treats all accesses to the same mapping variable as equivalent, regardless of which key is used. It cannot distinguish whether two functions reading and writing balances operate on the same entry or on entirely different ones. This prevents the analysis from tracking how a state change in one function affects the data that another function reads or guards, which is essential for detecting cross-function AC vulnerabilities. These two gaps motivate the ACPG’s extensions.

The first extension encodes guard semantics. A conditional branch qualifies as an *authorization guard* for a downstream operation when two conditions hold: its predicate depends, directly or transitively through data-flow edges, on the caller’s identity, and it dominates the operation, meaning every execution path from the function entry to that operation must pass through the branch.

In Solidity, the primary on-chain sources of caller identity are `msg.sender` and `tx.origin`. Our implementation tracks transitive dependencies from these two primitives via Slither’s data-flow analysis. Authorization guards take the form of inline `require` statements, `if` branches, or modifier bodies that encapsulate such checks. When both criteria are met, the ACPG records an E_{role} edge from the guard to the protected operation, allowing later stages to directly query whether a given operation is protected by an authorization guard.

The second extension resolves cross-function state connections at the mapping-key level. Mapping variables in DeFi contracts serve as shared accounting structures across multiple principals, yet a standard CPG treats every access to the same mapping identically, making `balances[msg.sender]` and `balances[attacker]` indistinguishable. AC analysis needs only to determine whether each access targets the caller’s own entry or a different principal’s. The ACPG answers this through a *symbolic key* attached to each mapping access, a tuple recording the provenance of each index

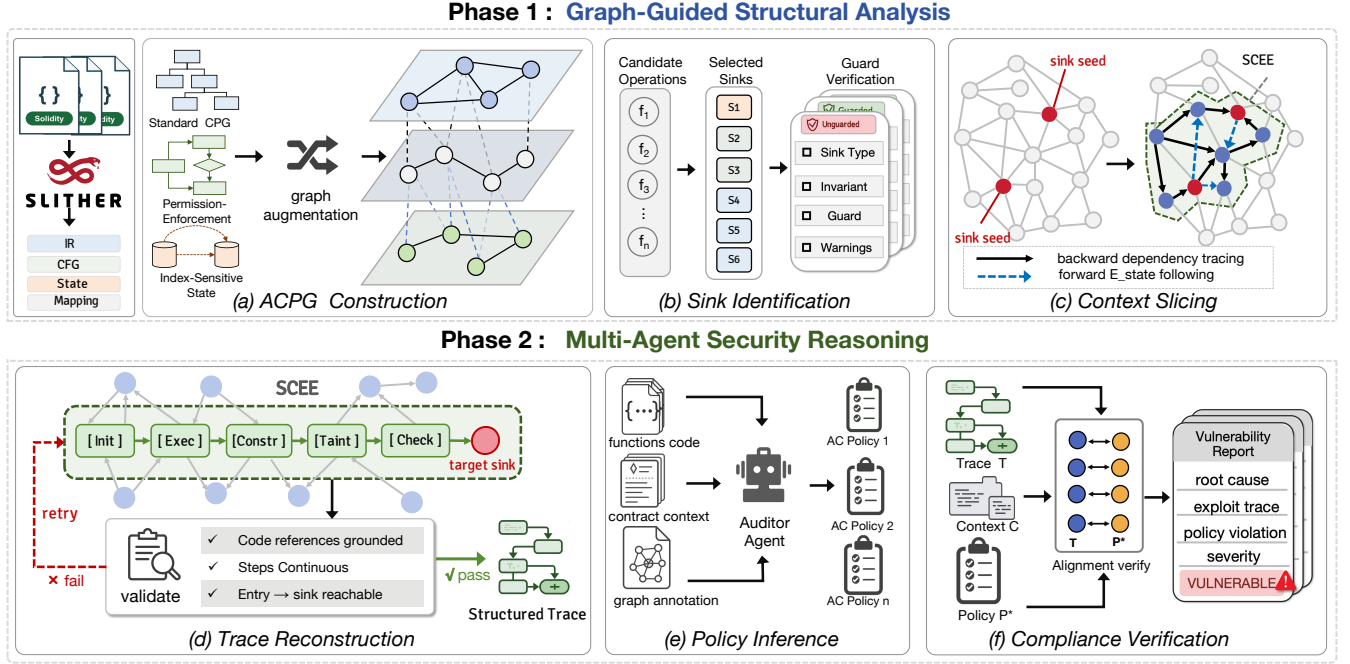


Figure 3: Overview of the ACLens framework. Phase 1 performs structural analysis guided by the program graph to construct the ACPG, identify critical sinks, and extract Sink-Conditioned Execution Environments (SCEEs). Phase 2 performs multi-agent security reasoning, where dedicated agents reconstruct sink-reaching traces, infer intended policies, and verify their alignment.

expression. For instance, `balances[msg.sender]` carries the key `(sender)`, while `allowances[msg.sender][spender]` with parameter `spender` carries `(sender, param:spender)`.

When a symbolic key contains parameter references, the analysis resolves each by following interprocedural call edges until a concrete origin (e.g., `msg.sender`) or a public entry point is reached. Write and read keys for the same variable are then compared position by position. Matching origins yield high confidence, distinct parameter sources yield medium, and unresolved locals yield low; incompatible origins (e.g., `sender` vs. a constant) produce no edge. The minimum confidence across positions scales the E_{state} edge weight as $w = 0.9 \times c$ ($c \in \{1.0, 0.67, 0.33\}$), so `balances[msg.sender]` in one function connects at weight 0.9 to the same entry elsewhere while unrelated accesses remain disconnected. Figure 2 illustrates these connections on XBridge (§2).

The resulting ACPG encodes the structural evidence that subsequent analysis builds on. Sink identification (§3.2.2) queries these edges to locate control variables and determine whether each sink is guarded, while context slicing (§3.2.3) uses the edge weights to extract code regions that preserve the security-relevant context for downstream reasoning.

3.2.2 AC-Critical Sink Identification. Given the ACPG, the next step is to determine *which* operations in a contract require authorization. We call an operation an *AC-critical sink* if an unauthorized execution would violate at least one contract-level security invariant. A security invariant here refers to a property the contract

should maintain throughout normal operation, whose violation leads to irreversible security consequences.

Definition 1 (AC-Critical Sink). Given an ACPG G_{AC} , an operation node $s \in V$ is an AC-critical sink if there exists a security invariant $I \in \{\text{GOVERNANCE}, \text{ASSET}, \text{AUTHORIZATION}\}$ such that an unprivileged caller performing s would violate I :

$$S_{crit} = \{s \in V \mid \exists I. \text{effect}(s, c_{\perp}) \models \neg I\} \quad (1)$$

Here $c_{\perp}$ models a caller that holds no privileged role in the contract, and $\text{effect}(s, c_{\perp})$ denotes the post-state that results from executing s under caller $c_{\perp}$, assuming s is reached regardless of any guards on the enclosing path. This identifies operations whose uncontrolled execution would be harmful; whether the guards currently in place enforce an adequate policy is assessed separately during policy inference (§3.3).

Analyzing documented DeFi incidents [??], we classify the damage from uncontrolled execution into three invariants. GOVERNANCE covers operations that alter the contract’s own access policy. ASSET covers external calls and value transfers that cause direct financial loss. AUTHORIZATION covers writes to shared state affecting other users’ funds or permissions, pervasive in DeFi’s mapping-based accounting but largely unaddressed by existing tools [????]. Table 1 lists the six sink categories derived from these invariants.

S1, S3, S4, and S5 correspond to operations recognized in prior AC analysis [???]. S2 and S6 are enabled by the ACPG extensions from §3.2.1. For S2, guard edges (E_{role}) provide a structural alternative to

Table 1: AC-critical sink taxonomy.

Invariant	Sink	Detection criterion
GOVERNANCE	S2. Policy-var. mutation	Write to variable read by a caller-identity guard
	S3. Destruction	selfdestruct instruction
	S5. Open initializer	Init entry without a caller-identity guard
ASSET	S1. External interaction	External call or value transfer
AUTH.	S6. Cross-user mutation	Mapping write with resolved key $\neq$ msg.sender
(inherited)	S4. Transitive closure	Reachable from S1–S3 or S6 via E_{call}

heuristic name matching [? ?]. A state variable read at the source of a guard edge is governance-relevant, so writes to it are classified as S2 sinks. For S6, key-resolved state edges (E_{state}) distinguish whether a mapping write targets the caller's own entry or another user's, capturing cross-user mutations that standard representations lack the precision to detect [?].

Before enumeration, we exclude entry points whose access is fixed by the execution model or limited to caller-local effects, such as constructors, fallbacks, and standard inherited token wrappers. After enumeration, we query the ACPG to determine whether a guard dominates each sink, provisionally marking it as *unguarded*, *guarded*, or *excluded*. Whether existing guards enforce the intended policy is assessed in §3.3.

3.2.3 Relevance-Guided Context Slicing. Classical program slicing [?] on a standard CPG may inadvertently exclude cross-function AC evidence [?]. Generic dependence edges do not encode permission semantics, and depth-bounded traversals may prune the guard conditions or state dependencies relevant to AC judgment. In the ACPG, these guard relations and cross-function state connections are already explicit. We therefore slice over an AC-aware dependency set, with raw control-flow structure no longer serving as the primary traversal basis.

SCEE construction. Given the sink set $\mathcal{S}_{crit}$, we perform relevance-guided backward slicing on the ACPG dependency edges $E_{dep} = E \setminus E_{CFG}$ to extract a *Sink-Conditioned Execution Environment* (SCEE) around each sink. CFG edges are excluded because they encode execution order rather than the dependency semantics needed for AC judgment.

Edge types contribute differently to AC judgment. Guard and modifier edges have the highest priority because they directly encode permission checks. State edges rank second, scaled by key-match confidence, followed by control-dependency, data-flow, and call edges. We formulate traversal as a maximum-product-path search on a priority queue. Each sink starts with relevance 1.0, and relevance decays along edges according to their type-specific weights, as defined in eq. (2).

$$rel(v) = \max_{(v,u) \in E_{dep}} [rel(u) \cdot w(v,u) \cdot \alpha], \quad rel(s)=1 \quad \forall s \in \mathcal{S}_f \quad (2)$$

where α is the decay factor (set to 0.85) and τ is the relevance threshold (set to 0.05). Nodes whose relevance falls below τ are not expanded further; the resulting SCEE comprises all nodes that survive this threshold. We also traverse E_{state} in both directions, because whether a state write is dangerous depends in part on how

Algorithm 1 Relevance-Guided Backward Slicing

Input: ACPG $G_{AC}=(V,E)$, sinks $\mathcal{S}_f$ of function f , decay α , threshold τ

Output: sink-conditioned execution environment $SCEE_f$

// $par(v,u)$: parallel edges between v and u in the multi-graph

```

1:  $E_{dep} \leftarrow \{e \in E \mid w(e) > 0\}$  // dependency edges only
2:  $rel[v] \leftarrow 0$  for all  $v \in V$ 
3:  $H \leftarrow$  empty priority queue;  $\mathcal{R} \leftarrow \emptyset$ 
4: for each sink  $s \in \mathcal{S}_f$  do
5:    $rel[s] \leftarrow 1.0$ ; push  $(1.0, s)$  into  $H$ 
6: while  $H \neq \emptyset$  do
7:    $(r_u, u) \leftarrow \text{EXTRACTMAX}(H)$ 
8:   if  $r_u < rel[u]$  then
9:     continue // superseded entry
10:   $\mathcal{R} \leftarrow \mathcal{R} \cup \{u\}$ 
11:  for each  $(v, u) \in E_{dep}$  do // backward: predecessors
12:     $r_v \leftarrow r_u \cdot \max_{e \in par(v,u)} w(e) \cdot \alpha$ 
13:    if  $r_v \geq \tau$  and  $r_v > rel[v]$  then
14:       $rel[v] \leftarrow r_v$ ; push  $(r_v, v)$  into  $H$ 
15:  for each  $(u, v) \in E_{state}$  do // forward: state readers
16:     $r_v \leftarrow r_u \cdot w(u, v) \cdot \alpha$ 
17:    if  $r_v \geq \tau$  and  $r_v > rel[v]$  then
18:       $rel[v] \leftarrow r_v$ ; push  $(r_v, v)$  into  $H$ 
19: return BUILDSCCE( $\mathcal{R}$ )

```

later code reads or guards that variable. Algorithm 1 formalizes the procedure.

The algorithm follows Dijkstra-style relaxation on a max-product semiring, computing optimal relevance in $O((|V|+|E|) \log |V|)$ time. With $\alpha=0.85$ and $\tau=0.05$, a path of weight-1.0 edges (guard, modifier, or role edges) retains relevance above τ for up to 18 hops ($0.85^{18} \approx 0.054$), while a path of call edges (weight 0.5) drops below τ at hop 4 ($0.425^4 \approx 0.033$). The weight assignment biases the SCEE toward nodes connected to the sink through edges that encode permission semantics, while nodes reachable only via low-weight indirect paths are excluded.

Running example. On XBridge (Figure 2), ACPG construction links the `_tokenOwner` write in `listToken` to the downstream read in `withdrawTokens` and marks `withdrawTokens` as protected by a guard dependent on caller identity. Sink identification marks this write as S2 (GOVERNANCE) and the transfer in `withdrawTokens` as S1 (ASSET). Guard verification then classifies `withdrawTokens` as GUARDED via `require(msg.sender == _tokenOwner[token])`. It classifies `listToken` as UNGUARDED. Starting from these sinks, relevance-guided slicing builds an SCEE that retains the full state chain from the `_tokenOwner` write to its guard read. Auxiliary operations in `listToken` with no dependency path to either sink, such as token metadata queries and event emissions, fall below τ and are excluded.

3.3 Multi-Agent Security Reasoning

Once phase one fixes the evidence boundary, the remaining question is whether the behavior allowed by the code matches an authorization policy that the contract never explicitly declares

(C2). Three distinct judgments are involved, namely identifying which execution paths reach the sink, determining the intended authorization constraints, and assessing whether the observed paths violate those constraints. A single model handling all three is prone to anchoring on its initial interpretation, propagating errors across judgments. Prior multi-agent designs [?] separate audit and review roles but give every agent the same full contract text, thereby discarding the structural narrowing from phase one.

Phase two assigns each judgment to a dedicated agent. The Trace Reconstructor (A_{trace}) reconstructs a candidate sink-reaching execution trace within the SCEE for each sink. The Auditor (A_{audit}) infers the intended authorization policy from code structure, graph annotations, and peer functions. The Verifier (A_{verify}) receives both outputs and judges whether the reconstructed trace reaches the sink without satisfying the inferred policy, and whether the resulting effect extends beyond the caller's own state. Section 4 quantifies the benefit of this decomposition.

3.3.1 Trace Reconstruction. AC vulnerabilities arise when critical operations lack adequate authorization constraints. In Solidity, authorization guards are predominantly enforced through modifiers or top-level require statements that execute before the function body, applying uniformly to all paths within the function. When such an entry-level guard is missing, the gap is observable along any path reaching the sink. Unlike reentrancy or front-running, which depend on specific execution orderings, AC defects in this dominant pattern are structural and path-independent.

A_{trace} exploits this property by reconstructing one candidate execution trace per sink, starting from the externally reachable entry point within the SCEE. Because entry-level guards apply uniformly to all paths, a single verified path that reaches the sink without encountering an adequate guard suffices to support a VULNERABLE judgment (§3.3.3 formalizes the sufficiency condition). The reconstruction prompt follows a five-step chain-of-thought protocol that advances from entry-state identification through forward execution, constraint accumulation, caller-identity propagation tracking, to end-to-end feasibility assessment.

The reconstructed trace is a structured sequence of verifiable assertions, each anchored to a code fragment in the SCEE. A deterministic check then confirms fragment existence, step continuity, and end-to-end reachability; failure triggers reconstruction. With the reasoning scope already narrowed by phase-one slicing, 89.1% of sinks pass validation in the first round. Figure 4 excerpts the core instructions.

3.3.2 Policy Inference. The Auditor infers an authorization policy $\mathcal{P}_f^* = (\mathcal{A}_f, \Phi_f)$ for each target function f , where $\mathcal{A}_f \in \{\text{Public}, \text{Owner}, \text{Role}, \text{DAO}\}$ specifies the required caller class and Φ_f lists the state preconditions that must hold before execution. The granularity is per function rather than per sink, because Solidity guards are enforced at function entry (§3.3.1) and all sinks within the same function therefore share the same authorization boundary. Within a single contract, developers typically protect operations with similar security implications in a similar way. Deviations from such recurring patterns have been shown to be a reliable signal for latent defects in prior work on inconsistency-based bug detection [?], and SPCon [?] confirmed this regularity for access

Prompt for Trace Reconstructor

[Input] SCEE source {scee}, target sinks {sinks}

[Task] For each sink in {sinks}, reconstruct a structured execution trace from the externally reachable entry point to the sink within the evidence boundary {scee}.

[Methodology] For each sink, perform five steps:

1. *State Init* — Caller α is unconstrained; state variables hold defaults.
2. *Forward Exec* — Traverse entry $\rightarrow$ sink, inlining modifiers and internal calls.
3. *Constraint Accum* — Record every require/if/assert as path condition π .
4. *Identity Propagation* — Track msg.sender flow into π or sink operands.
5. *Feasibility* — Verify π is satisfiable under σ_0 .

[Output] Each step: {position, type $\in$ {MODIFIER, CONSTRAINT, STATEWRITE, EXT CALL}, code_ref, formula}

Figure 4: The prompt for the Trace Reconstructor agent.

control in smart contracts. The Auditor exploits this regularity by comparing the target function against structurally similar peer functions selected from the ACPG. When comparable functions that write to the same governance variable or perform the same category of sink operation are consistently guarded, an unguarded target performing a similar operation is more likely a missing restriction than a deliberate design choice.

A_{audit} receives four inputs (Figure 5): the target function's source, contract-level context (state variable declarations and modifier definitions), a ranked set of peer functions from the same contract, and the graph annotations produced by phase one (per-sink guard status, control-variable alerts, and cross-user write flags from §3.2.2). Peers are selected from the same contract based on structural overlap with the target. The ACPG makes three forms of overlap observable between functions: (1) shared writes to variables in $\mathcal{V}_{crit}$ (sink operands $\cup$ control variables from §3.2.2), (2) sinks classified under the same security invariant (Table 1), and (3) shared modifiers or guard predicates referencing the same role variable. Candidates with more overlapping signals rank higher. When two or more top-ranked peers enforce caller identity guards while the target lacks equivalent protection, the asymmetry constitutes evidence of a missing restriction. When all comparable peers are uniformly permissionless or scoped to the caller's own state, the target is presumed open by design. If fewer than two peers are available or their patterns conflict, the Auditor lowers its confidence and relies primarily on graph annotations as supplementary evidence.

3.3.3 Compliance Verification. Given the candidate traces from A_{trace} and the inferred policy from A_{audit} , the Verifier determines their alignment. For a given sink s with locally validated trace set $\mathcal{T}_s$ and inferred policy $\mathcal{P}_f^*$, we write $\text{NonLocal}(T)$ for traces whose effects are not confined to caller-local state. The positive judgment is expressed in eq. (3).

$$\text{VULNERABLE}(s) \iff \exists T \in \mathcal{T}_s : T \not\models \mathcal{P}_f^* \wedge \text{NonLocal}(T) \quad (3)$$

where $T \not\models \mathcal{P}_f^*$ means that the trace violates the inferred caller class or state preconditions for f . $\text{NonLocal}(T)$ holds when the trace affects shared or protocol-level state rather than only caller-local

Prompt for Auditor	
[Input]	Target function {code}, contract context {context}, peer functions {peers}, graph annotations {annotations}
[Task]	Infer the intended access control policy $\mathcal{P}_f^* = (\mathcal{A}_f, \Phi_f)$ for {code}: (1) required caller class (<i>Public</i> , <i>Owner</i> , <i>Role</i> , <i>DAO</i>); (2) state preconditions that must hold. Use {context} to resolve types and roles referenced in {code}. Use {annotations} as supplementary structural evidence.
[Peer Comparison]	If ≥ 2 comparable peers consistently enforce caller identity guards while the target lacks equivalent protection, this asymmetry is evidence of a missing restriction. Uniformly permissionless peers indicate the target is permissionless by design. Fewer or conflicting peers lower confidence but do not preclude inference; use {annotations} as supplementary evidence.
[Output]	{caller_class, state_preconditions, supporting_peers, protection_pattern, confidence}

Figure 5: The prompt for the Auditor agent.

Prompt for Verifier	
[Input]	Inferred policy {policy}, reconstructed trace {trace}, contract context {context}
[Task]	Given {context}, determine whether {trace} violates {policy} and whether the effect extends beyond the caller's own state.
[Judgment]	Flag a policy violation if {trace} reaches a sensitive operation without satisfying the required caller class or state preconditions. Classify as VULNERABLE only when the violation affects shared or protocol-level state (governance variables, cross-user mapping entries, external asset transfers). Caller-local operations are SAFE .
[Output]	{status $\in$ {VULNERABLE, SAFE, UNKNOWN}, reasoning, severity}

Figure 6: The prompt for the Verifier agent.

entries. This includes writes to governance and configuration variables, to aggregate accounting state, and to mapping entries whose effective principal is not bound to `msg.sender` or to a parameter already constrained to equal `msg.sender`. The second conjunct ensures that ACLens does not flag caller-only self-service behavior as a vulnerability. A single verified violating trace suffices for a positive finding; exhausting remaining branches is unnecessary. When neither condition triggers, *Averify* conservatively outputs UNKNOWN. Constraining the Verifier to trace-policy alignment also prevents it from re-inferring the policy or hypothesizing attack scenarios unsupported by its inputs. Figure 6 excerpts the core instructions.

Running example. On XBridge, the Verifier aligns the trace (register on a new chain pair $\rightarrow$ overwrite `_tokenOwner` $\rightarrow$ withdraw) against the Auditor's policy and judges that the path both violates the intended policy and affects protocol-controlled assets, yielding VULNERABLE.

4 Evaluation

We evaluate ACLens by answering four research questions.

RQ1: How does ACLens perform in detecting AC vulnerabilities compared to existing approaches?

RQ2: How well does ACLens generalize across different LLM backbones?

RQ3: What is the contribution of each design component to the overall detection result?

RQ4: How much does structural analysis reduce the analysis context, and what cost does the resulting reduction enable?

4.1 Experimental Setup

All LLM agents return JSON-formatted outputs. We set temperature to 0 and fix the random seed, following standard practice in LLM-based vulnerability detection [?]. Context repair runs for at most $k=2$ rounds. Each benchmark project is processed end-to-end without manual intervention. Under these settings the pipeline produces identical outputs for identical inputs; we therefore report single-run results. All experiments ran on a MacBook Pro with Apple M3 Max.

Metrics. Following ACTaint [?] and AChecker [?], we report TP, FP, FN, Recall R , and F1. We adopt a strict *function-level* evaluation protocol. For function-level evaluation on the DeFi set, a reported function counts as TP only when it independently exposes an exploitable root cause consistent with the incident; otherwise it counts as FP, and unmatched benchmark functions count as FN.

4.2 Benchmarks

Our evaluation uses two complementary benchmarks: **CVE**, the 15-contract CVE dataset widely adopted in prior work [? ? ?], and **DeFi**, a benchmark we construct from confirmed on-chain DeFi AC exploits from 2017 to 2024. CVE covers classic single-function patterns on which existing tools already achieve high recall (ACTaint: 93.3%). The DeFi dataset targets production DeFi contracts with cross-function authorization logic, where AC exploits cause the largest share of financial losses [?] yet remain largely undetected. **DeFi.** We screen all incidents in DeFiHackLabs [?] through 2024 and retain those whose root cause is an AC vulnerability and whose source code is verified on-chain. This yields 54 confirmed exploits across six chains, covering 750 contract, library, and interface definitions totaling 85.6 KLoC. We then exclude 8 cases that are not genuine AC flaws, such as Merkle-guarded mints and intentionally permissionless routers. The remaining set contains 46 validated projects.

To establish function-level ground truth, two researchers with over two years of smart contract auditing experience independently reviewed each project. For every case they traced the on-chain exploit transaction, identified the root-cause function(s), and annotated all functions whose unguarded state independently enables the exploit. The first annotation round produced 66 functions on which both annotators agreed. Disagreements and newly identified entry points were resolved through joint re-examination of exploit traces and contract state transitions, producing a final set of 98 vulnerable functions across the 46 projects.

4.3 Baselines

We compare ACLens against six baselines spanning the main technical directions in AC detection. The static tools cover pattern matching, symbolic execution, and bytecode dataflow analysis; the two LLM tools represent the closest prior work in agent-guided AC

analysis and a leading general-purpose LLM auditor, respectively. For the main comparison in RQ1, all LLM-based methods use GPT-4o, so performance differences reflect methodology rather than model capability.

Static analysis and symbolic execution. **Slither** [?] is an industry-standard pattern-based analyzer. We count findings from its AC-related detectors. **Mythril** [?] performs symbolic execution on EVM bytecode. **SPCon** [?] mines role-permission relationships from bytecode via frequent-itemset analysis. **AChecker** [?] traces msg.sender dependencies through bytecode-level data flow with symbolic filtering.

LLM-based detection. **ACTaint** [?] is a dual-agent system guided by a Critical Variable Dependency Graph (CVDG) for LLM-based taint analysis. **GPTLens** [?] uses an auditor-critic pair with shared context. We adapt its auditor prompt by replacing the general vulnerability taxonomy with AC-specific categories while preserving the original auditor-critic interaction structure.

ACTaint and GPTLens are reproduced from their published descriptions. We validate our ACTaint reproduction against the originally reported CVE recall and obtain the same result (14/15).

All baselines are evaluated on the DeFi dataset with the same function-level evaluation protocol.

4.4 Results

4.4.1 RQ1: Detection Effectiveness. We first evaluate ACLens on CVE, the 15-contract CVE benchmark widely adopted in prior AC detection work [? ? ?]. ACLens detects all 15 cases, followed by ACTaint (14/15), AChecker (12/15), SPCon (9/15), GPTLens (5/15), Mythril (4/15), and Slither (1/15). The case that ACTaint misses involves an external function that directly modifies a critical state variable without any msg.sender dependency in its control flow. ACTaint's taint agent searches for a propagation path from an untrusted source to the sink and, finding none, classifies the function as safe. ACLens identifies the function as an unguarded sink during phase one and flags it without requiring a taint path.

Table 2 presents the DeFi dataset results. The benchmark contains 98 vulnerable functions across 46 projects; one project fails Slither compilation for all tools, leaving 97 analyzable functions.

ACLens detects 81 of the 97 functions (Table 2), compared with 36 for ACTaint, the strongest baseline. We inspected the cases on which the two systems disagree. The majority of additional detections by ACLens involve completely unguarded entry points that modify critical state or trigger external calls. Because these functions contain no msg.sender check, ACTaint's taint agent finds no propagation path to evaluate and does not flag them. ACLens catches them because sink identification in phase one flags unguarded critical operations independently of taint reachability; the XBridge vulnerability (§2) illustrates this pattern at the cross-function level. A smaller group of additional detections involves functions with partial guards that ACTaint considers sufficient but that ACLens's policy inference judges incomplete based on cross-function state dependencies visible in the ACPG.

GPTLens detects 18 functions but produces 43 false positives. Without structural guidance, its auditor receives the full contract source and frequently hallucinates non-existent guards or dismisses missing protections as intentional. The four non-LLM tools achieve

Table 2: Detection performance on the DeFi dataset

Method	TP	FP	FN	P(%)	R(%)	F1(%)
Slither	8	22	89	26.7	8.2	12.6
Mythril	0	0	97	—	0.0	—
SPCon	1	0	96	100.0	1.0	2.0
AChecker	0	0	97	—	0.0	—
GPTLens	18	43	79	29.5	18.6	22.8
ACTaint	36	72	61	33.3	37.1	35.1
ACLens	81	116	16	41.1	83.5	55.1

Table 3: Performance comparison across different LLM backbones

Tool	Model	TP	FP	P(%)	R(%)	ΔP	ΔR
ACLens	GPT-4o	81	116	41.1	83.5		
ACLens	GPT-4o-mini	79	119	39.9	81.4	−1.2	−2.1
ACLens	DeepSeek-R1	80	115	41.0	82.5	−0.1	−1.0
ACTaint	GPT-4o	36	72	33.3	37.1		
ACTaint	GPT-4o-mini	42	427	9.0	43.3	−24.3	+6.2
ACTaint	DeepSeek-R1	39	118	24.8	40.2	−8.5	+3.1

near-zero recall for distinct reasons. Mythril's symbolic execution encounters path explosion or analysis timeouts on complex DeFi contracts, and its SWC detectors do not target AC vulnerabilities. Slither's built-in AC rules target misuse of existing guards (e.g., unprotected selfdestruct) rather than missing guards, capturing only 8 of 97 functions. AChecker performs symbolic execution on EVM bytecode, but the complex control flow and extensive external calls in DeFi contracts cause frequent analysis timeouts. SPCon infers permission policies by mining on-chain transaction histories [?], yet many vulnerable DeFi functions lack sufficient transaction records for role mining, leaving SPCon unable to derive a policy.

Answer for RQ1: ACLens detects all 15 CVE cases and 81 of 97 DeFi dataset functions, outperforming the strongest baseline by 46.4 recall points and 20.0 F1 points. The additional detections are predominantly unguarded entry points where no taint path exists for prior tools to evaluate.

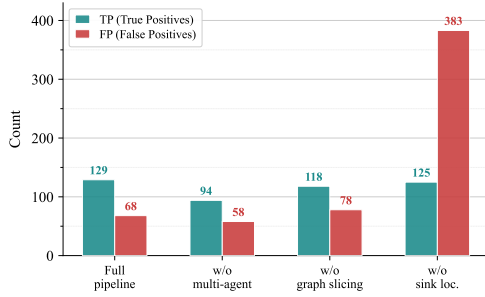
4.4.2 RQ2: Phase-Two Backbone Sensitivity. To answer RQ2, we examine whether detection performance remains stable when the LLM backbone changes. We keep the pipeline structure and prompts fixed and replace only the backbone, running both ACLens and ACTaint with GPT-4o, GPT-4o-mini, and DeepSeek-R1 [?].

Table 3 shows that ACLens varies little across backbones, with recall staying within a 2.1 pp band and false positives between 115 and 119. ACTaint fluctuates far more. Under GPT-4o-mini it recovers six additional TPs but false positives rise from 72 to 427, dropping precision to 9.0%; DeepSeek-R1 shows the same trend at smaller magnitude (FP = 118, P = 24.8%).

Because phase one of ACLens fixes critical functions and evidence scope before any LLM call, backbone quality affects only the interpretation of a focused context. ACTaint delegates both

Table 4: Performance comparison in the ablation study

Configuration	TP	FP	FN	P(%)	R(%)	F1(%)
Full two-phase system	81	116	16	41.1	83.5	55.1
w/o multi-agent	65	87	32	42.8	67.0	52.2
w/o graph slicing	77	119	20	39.3	79.4	52.6
w/o sink identification	81	429	16	15.9	83.5	26.7

**Figure 7: TP and FP counts in the ablation study**

scoping and judgment to the LLM, compounding errors when the backbone weakens. Practically, ACLens can switch to GPT-4o-mini while losing only 2 TPs.

Answer for RQ2: Swapping the backbone moves ACLens’s recall by at most 2.1 percentage points and its false-positive count by at most 4. ACTaint fluctuates much more, with false positives rising from 72 to 427 under GPT-4o-mini.

4.4.3 RQ3: Phase Contribution. To answer RQ3, we build three ablation variants and evaluate them on the DeFi dataset with GPT-4o. *w/o multi-agent* collapses phase two into a single LLM call that still receives the SCEE and graph annotations from phase one. *w/o graph slicing* retains phase-two reasoning but replaces the scoped SCEE with the full contract source. *w/o sink identification* disables phase one’s sink filter and runs the full system on every public/external function.

Removing multi-agent reasoning causes the largest recall drop. TP decreases from 81 to 65 and recall falls from 83.5% to 67.0%. The 16 newly missed cases concentrate on multi-hop privilege escalation chains. When trace construction and policy judgment share a single prompt, the model tends to rationalize missing guards as intentional design rather than flag them as violations.

Without graph slicing, the LLM receives the full contract source instead of the scoped SCEE. TP drops from 81 to 77 and recall decreases to 79.4%. All four newly missed functions belong to projects exceeding 1,500 lines, where the AC-relevant statements are buried among hundreds of unrelated functions and the LLM fails to locate the missing guard.

Removing sink identification has no effect on recall (TP stays at 81) but inflates false positives from 116 to 429 (Figure 7), cutting F1 from 55.1% to 26.7%. The additional FPs come from functions that contain no security-relevant operations; their lack of a guard is benign by design, and only the sink filter prevents them from entering the analysis pipeline.

Table 5: Cost profile for the five largest DeFi dataset projects

Representative function	LoC	Tokens (K)	Time (s)	Cost (€)
approveToken	4,053	109.3	27	2.0
swap	4,008	34.6	23	0.7
transferFrom	3,367	42.4	20	0.8
setToken	3,070	86.7	29	1.9
withdraw	2,488	82.8	19	1.6
All 46 (mean)	1,375	59.0	22	1.2

In summary, removing any of the three components drops F1 below the full system’s 55.1%, but the failure modes differ. Sink identification controls false positives, graph slicing preserves recall on large contracts, and multi-agent separation preserves recall on multi-hop privilege chains.

Answer for RQ3: Each component addresses a distinct failure mode. Removing multi-agent reasoning drops recall by 16.5 pp; removing graph slicing drops recall by 4.1 pp on large contracts; removing sink identification inflates false positives from 116 to 429. All three variants fall below the full system’s 55.1% F1.

4.4.4 RQ4: Scoping Efficiency and Cost. To answer RQ4, we measure how graph-guided slicing changes the input that reaches phase two. Across all 248 analyzed functions in the 46 DeFi dataset projects, SCEEs average **92 whitespace tokens** per function (roughly 120 LLM subword tokens), versus 6,333 tokens for the full contract source, a 97.9% average reduction. Most of a contract’s code is simply irrelevant to any single AC decision.

Table 5 profiles analysis cost under GPT-4o-mini (\$0.15/1M input, \$0.60/1M output tokens) on the five largest DeFi dataset projects.

Across all 46 projects, the mean cost is 1.2 € per project (max 7.5 €), consuming 59.0K tokens on average with a mean wall-clock time of 21.9 s. Cost tracks the number of critical functions that survive phase one more closely than raw code volume.

Answer for RQ4: Graph-guided slicing reduces the per-function input from 6,333 tokens to 92 tokens on average (97.9%). Under GPT-4o-mini, the mean analysis cost is 1.2 €/project, and that cost tracks the number of surviving critical functions rather than raw code volume.

4.5 Error Analysis

We manually reviewed all false negatives and false positives on the DeFi dataset to identify the root causes of remaining detection gaps.

False negatives. We identified three categories among the 16 missed functions.

Six encode authorization semantics within business logic rather than in an explicit guard. For example, a DEX swap function may rely on slippage bounds and price oracle checks as its only protection. Because the authorization constraints appear as branch-level business conditions rather than entry-level caller checks, the Verifier cannot construct sufficient evidence to confirm a violation and returns UNKNOWN (\$3.3.1).

Five functions carry canonical ERC-20 signatures (burn, transfer, transferFrom) but implement non-standard logic such as tax mechanisms or rebasing [?]. In the DeFi dataset, standard-name functions with canonical semantics outnumber such variants 12:1, so the system's prior strongly favors classifying them as benign.

The remaining five are engineering gaps: three attribution mismatches that become TPs under contract-level evaluation, and two Slither compilation failures.

False positives. We categorized the 116 false positives into three groups.

The largest group (55 cases, 47%) involves intentionally permissionless operations such as deposit, withdraw, and liquidateBorrow, which are open by design in DeFi lending protocols. This reflects insufficient domain priors rather than a structural scoping error.

The second group (40 cases, 34%) consists of locally coherent but globally infeasible traces. The reconstructed execution path satisfies all branch conditions within the analyzed scope but violates constraints visible only at the whole-contract level, such as multi-day governance timelocks or mutually exclusive preconditions.

The remaining 21 cases (18%) are hallucinated dependencies where the LLM fabricates state connections absent from the contract [?]. The multi-agent design reduces but does not fully eliminate this failure mode (Table 4).

5 Discussion

5.1 Threats to Validity

External validity. The DeFi dataset covers 46 validated projects across six chains and multiple protocol types (DEXes, lending, yield aggregators, token contracts) spanning 2017–2024, and 100% recall on the CVE dataset confirms no regression on simpler patterns. The benchmark remains dominated by missing-modifier flaws in DeFi protocols, however, and generalization to DAO governance, cross-chain bridges, and non-Solidity languages (Vyper, Move) remains to be validated.

Construct validity. Recall and precision weight all vulnerabilities equally regardless of severity [?]. The strict function-level protocol (Section 4) mitigates this by requiring both location and root-cause match, and DeFi includes only confirmed exploits with real financial losses [?].

Data leakage. Some contracts may appear in model pretraining corpora, but the two-phase design exposes only scoped fragments from deterministic analysis rather than full sources, limiting how much memorization can help.

5.2 Future Work

Domain-aware policy inference. Over half the false positives arise from intentionally permissionless DeFi primitives (self-service deposits, standard token operations, Compound-style lending). A retrieval-augmented knowledge base of known permissionless idioms, or a classifier trained on curated DeFi function signatures, could sharpen the boundary between deliberate openness and a genuinely missing guard.

Symbolic path feasibility. Attack paths with globally unsatisfiable constraints, such as multi-day timelocks or mutually exclusive preconditions, form the second major source of false positives. A symbolic back-end could help here, but the main challenge is not

only solving constraints; trace steps, guard conditions, and state interactions must first be translated into sound SMT formulas. SMT-based feasibility checking therefore remains future work.

6 Related Work

Static and AC-specialized analysis. Automated analysis of smart contracts has been studied through symbolic execution [?], pattern matching [?], Datalog reasoning [?], fuzzing [?], and formal verification [?]. Large-scale evaluations on tens of thousands of contracts show that these tools are effective at detecting arithmetic and reentrancy vulnerabilities but less accurate on access control, denial of service, and front-running [?]. Even under dedicated AC evaluation, detection remains challenging [?]. Several tools have been developed specifically for this problem. SPCon [?] mines role-permission patterns from on-chain transaction histories. AChecker [?] traces msg.sender dependencies through intraprocedural data flow on bytecode. Ethainter [?] uses Datalog-based taint analysis to track information flow from callers to sensitive operations. SoMo [?] matches modifier structures at the AST level. These tools have advanced AC detection from different angles, each targeting a specific class of authorization patterns.

LLM-based smart contract analysis. Recently, several studies have explored using LLMs for smart contract vulnerability detection. GPTScan [?] maps vulnerabilities to predefined templates and validates matches via static confirmation. GPTLens [?] employs an auditor–critic dialogue where both agents share the full contract as context. ACTaint [?] combines a variable-level dependency graph (CVDG) with dual LLM agents, where a sink agent identifies critical variable modifications and a taint agent traces data flow from untrusted sources to sinks. The CVDG tracks dependencies at the variable level, without distinguishing accesses to different entries of the same mapping. Broader evaluations have found that cross-function reasoning remains a persistent bottleneck for LLM-based contract analysis [?], and that ChatGPT-style auditing still exhibits unstable precision on real-world contracts [?]. iAudit [?], PropertyGPT [?], and SmartInv [?] target per-function auditing or property generation rather than AC detection. In the broader software engineering domain, agent role separation has been shown to reduce hallucination in collaborative code reasoning [?].

7 Conclusion

In this paper, we proposed ACLens, a multi-agent framework guided by program graphs for AC vulnerability detection. The first phase scopes the code that carries authorization semantics, while the second phase infers the intended policy and checks whether feasible traces violate it. Our evaluation on two benchmarks shows that this decomposition improves detection effectiveness on real DeFi access control cases across multiple LLM backbones and ablation settings. The main remaining difficulty lies in domain-specific policy understanding rather than structural evidence recovery. These results suggest that reliable AC detection hinges not on showing the model more code, but on first delimiting the evidence that actually carries authorization semantics.

Data Availability

The anonymous replication package, including the analysis framework, curated datasets, and experiment outputs, is publicly available at <https://doi.org/10.5281/zenodo.19220416>.

References

1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218

1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276